

Fondamenti di Informatica T2

Introduzione a Eclipse

Corso di Laurea in Ingegneria Informatica

Anno accademico 2023/2024

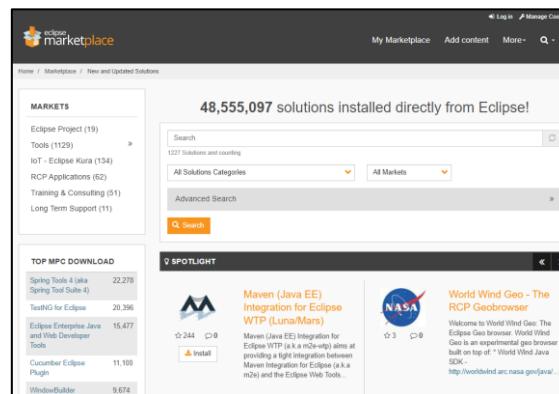
Prof. ROBERTA CALEGARI

Prof. AMBRA MOLESINI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)

Eclipse (1/3)

- Eclipse è un ambiente di sviluppo:
 - **opensource**: è scritto esso stesso in Java
 - **multiplatforma**: Windows, Mac, Linux,...
 - **multilinguaggio**: non serve solo per Java
 - **estendibile** tramite *plugin*
- ne esistono *migliaia* nel *MarketPlace*
<http://marketplace.eclipse.org>



MARKETS

- Eclipse Project (19)
- Tools (1129)
 - Application Development Frameworks (180)
 - Application Management (34)
 - Application Server (28)
 - BIRT (11)
 - Build and Deploy (90)
 - Business Intelligence, Reporting and Charting (13)
 - Code Management (90)
 - Collaboration (33)
 - Database (42)
 - Database Development (29)
 - Database Persistence (18)
 - Documentation (46)
 - EclipseRT Target Platform Components (12)
 - Editor (302)
 - Entertainment (15)
 - General Purpose Tools (106)
 - Graphics (35)
 - IDE (293)
 - Internet of Things (IoT) (33)
 - J2EE Development Platform (31)
 - J2ME (3)
 - Languages (121)
 - Linux Tools (16)
 - Logging (16)
 - Mobile and Device Development (45)
 - Modeling (86)
 - Modeling Tools (129)
 - Mylyn Connectors (15)
 - Network (11)

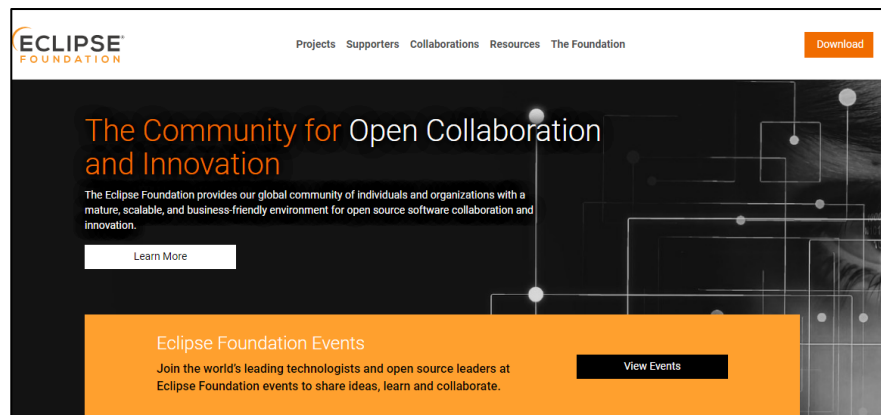


Eclipse (2/3)

- Eclipse è esso stesso un'applicazione Java
 - parte utilizzando il JDK/JRE impostato nel path di sistema
- Possiede il suo **compilatore autonomo**
 - non utilizza quello del JDK
 - può compilare *anche per versioni precedenti* di Java
- Richiede però necessariamente un **JRE/JDK esterno** per far **girare le applicazioni**
 - il default è la versione di Java (JDK o JRE) con cui è partito
 - ma può usare anche altre versioni di Java presenti sul PC

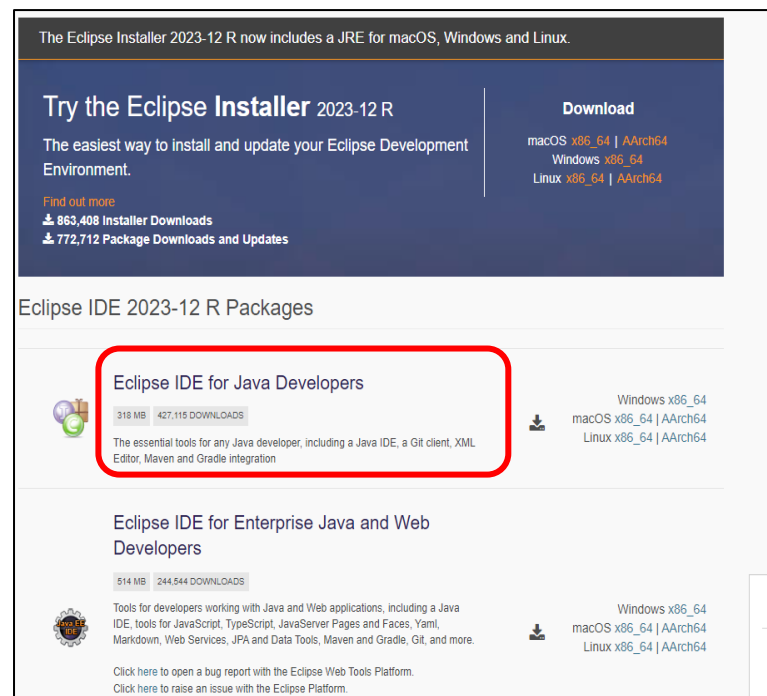
Eclipse (3/3)

- Attualmente viene rilasciata una nuova versione in simultanea con l'uscita delle nuove versioni di Java
 - in passato non era così
- Attualmente le versioni si chiamano *anno-mese*
 - ad esempio, 2023-12
 - in passato abbiamo avuto
 - astri, pianeti e lune varie: Callisto, Europa, Ganymede, Galileo, Helios, Indigo, Juno, Kepler, Luna, Mars
 - gas e altri nomi scientifici: Neon, Oxygen, Photon



La giusta versione

- Eclipse è una piattaforma configurabile
 - arbitrariamente arricchibile con *plugin*
 - al download sono perciò proposte le *configurazioni di più largo uso*
- Voi volete la
Eclipse IDE for Java Developers



The Eclipse Installer 2023-12 R now includes a JRE for macOS, Windows and Linux.

Try the Eclipse Installer 2023-12 R
The easiest way to install and update your Eclipse Development Environment.

Download
macOS x86_64 | AArch64
Windows x86_64
Linux x86_64 | AArch64

Eclipse IDE 2023-12 R Packages

Eclipse IDE for Java Developers
318 MB | 427,115 DOWNLOADS
The essential tools for any Java developer, including a Java IDE, a Git client, XML Editor, Maven and Gradle integration

Eclipse IDE for Enterprise Java and Web Developers
514 MB | 244,544 DOWNLOADS
Tools for developers working with Java and Web applications, including a Java IDE, tools for JavaScript, TypeScript, JavaServer Pages and Faces, YAML, Markdown, Web Services, JPA and Data Tools, Maven and Gradle, Git, and more.

Click here to open a bug report with the Eclipse Web Tools Platform.
Click here to raise an issue with the Eclipse Platform.

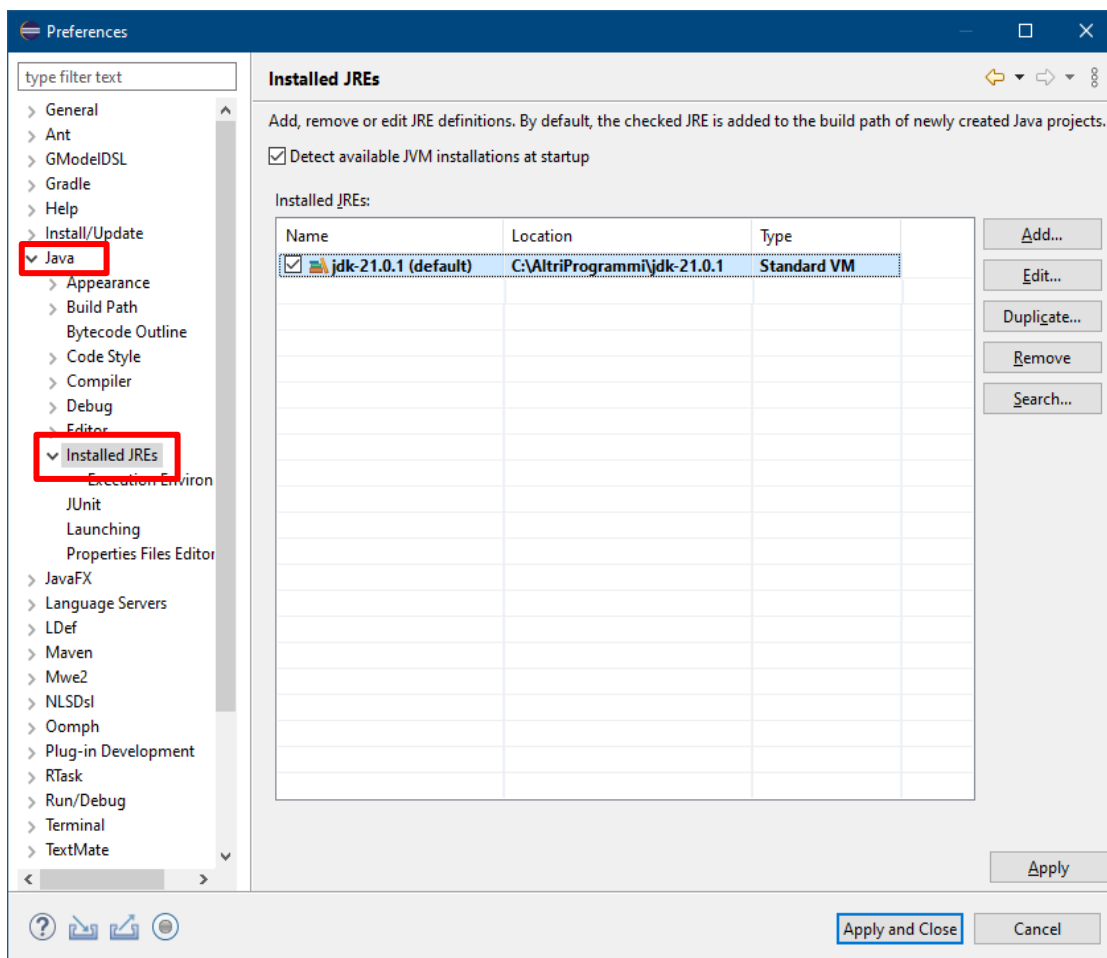


Installazione (1/2)

- Easy way: scaricare l'installer ed installare
Eclipse IDE for Java Developers
- Oppure, per chi ama il fai da te:
 1. Scaricare Eclipse IDE for Java Developers dal sito www.eclipse.org
 - prestare attenzione alla piattaforma utilizzata:
un JDK a 32 bit richiede Eclipse a 32 bit,
un JDK a 64 bit richiede Eclipse a 64 bit,
 2. Decomprimere in una cartella a scelta
 - meglio evitare C:\Programmi (questioni di diritti..)
 - **decomprimere = installare:** Eclipse NON scrive nulla nel registro di Windows, non semina file nel vostro disco: è tutto contenuto nella cartella decompressa
 - **opportuno creare a mano un collegamento sul desktop e/o nel menù Start**

Installazione (2/2)

- Per verificare a quale versione di Java è agganciato:
 - > Window
 - > Preferences
 - > Java
 - > Installed JREs
- Si possono aggiungere altri JRE che siano presenti sul Sistema (tasto Add)





Concetti chiave

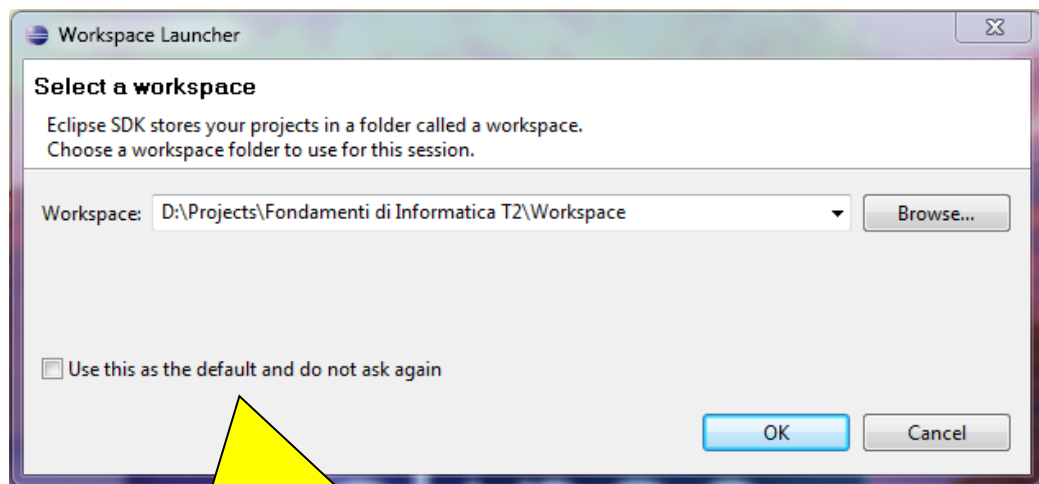
- Eclipse si basa su alcuni concetti chiave:
 - **workspace** lo spazio di lavoro
 - **progetto** l'entità concettuale su cui si lavora
- a loro volta strutturati in:
 - workspace → *prospettive, viste, editor, toolbar*
 - progetto → *file sorgenti, percorsi, cartelle, test, librerie e documentazione*
- Fin dall'avvio vi chiederà in che *workspace* lavorare
 - ne potete avere molti
 - organizzate con intelligenza il vostro lavoro per argomenti

Workspace

- Dal punto di vista LOGICO, un **workspace** è *l'ambiente di lavoro* in cui vengono creati e mantenuti i progetti
 - possiamo pensare al workspace come ad una raccolta di progetti
- Dal punto di vista FISICO, un **workspace** è una *cartella* in cui sono mantenute *le impostazioni e le caratteristiche* del workspace
 - ogni workspace ha le sue impostazioni *anche a livello Eclipse*
 - workspace diversi possono usare compilatori diversi, essere configurati in modo diverso, avere proprietà diverse, etc
 - i progetti inclusi logicamente nel workspace possono anche non risiedere in questa cartella: l'appartenenza di un progetto a un workspace è un fatto *LOGICO*, non fisico
 - perciò, un progetto può appartenere anche a più workspace

Primo avvio (1/3)

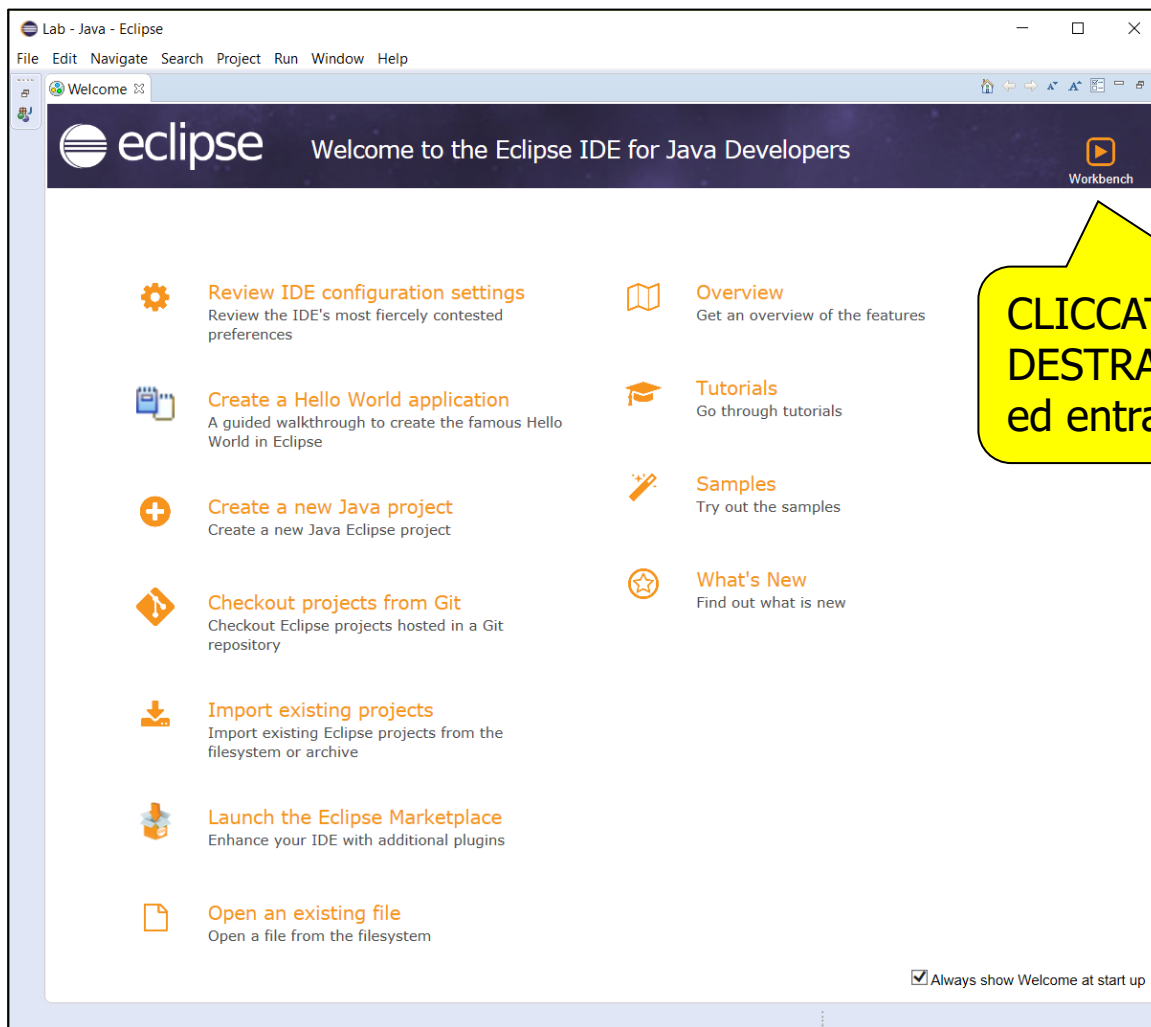
Ad ogni avvio, Eclipse vi chiederà in che workspace volete lavorare:



Impostare la cartella su cui salvare il workspace.

SOLO SE PENSATE DI USARE UN SOLO WORKSPACE, impostare questa scelta come default. Al prossimo avvio non verrà più mostrata questa schermata (sarà comunque possibile cambiare workspace da menù).

Primo avvio (2/3)



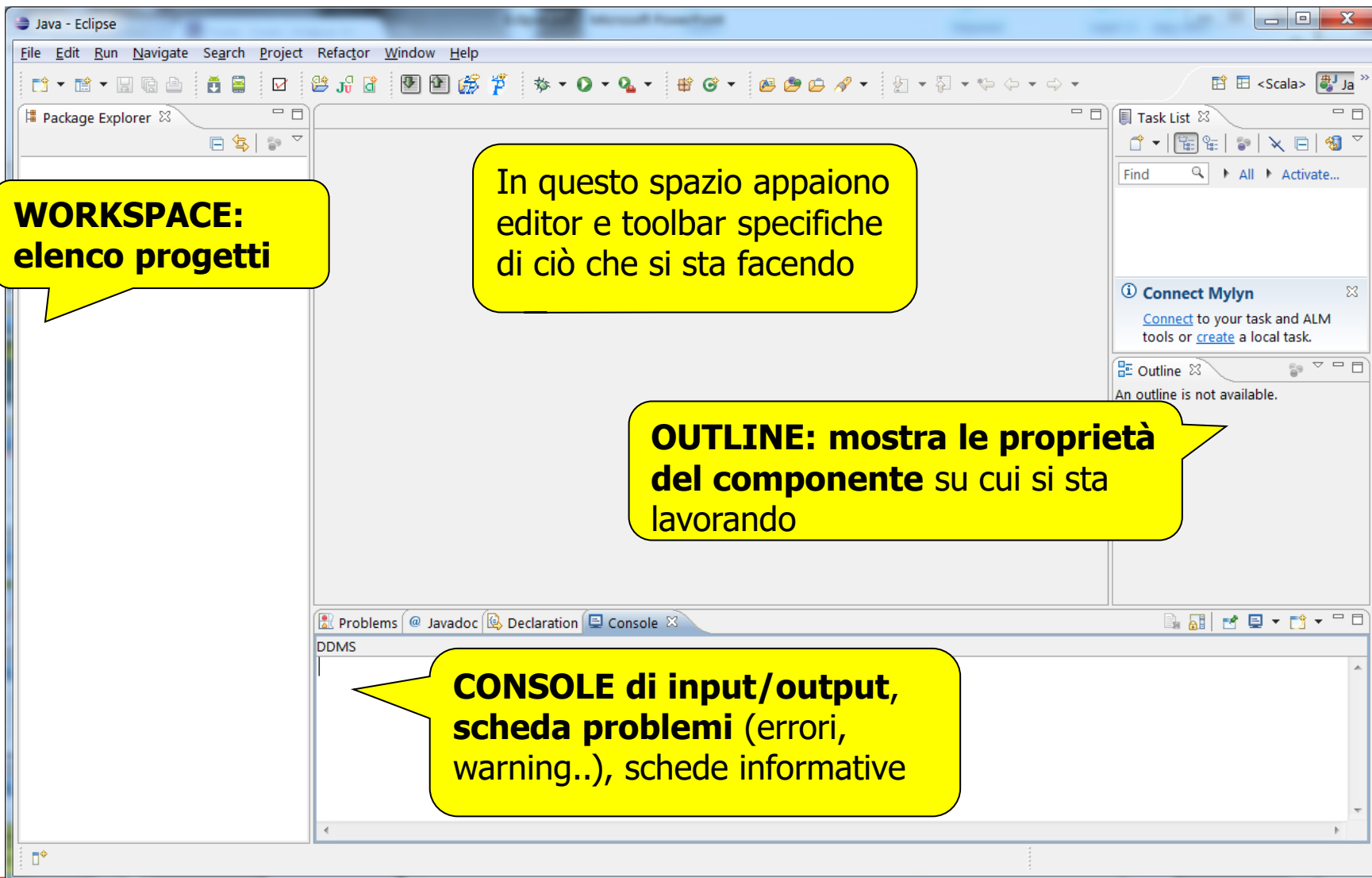
Primo avvio (3/3)

WORKSPACE:
elenco progetti

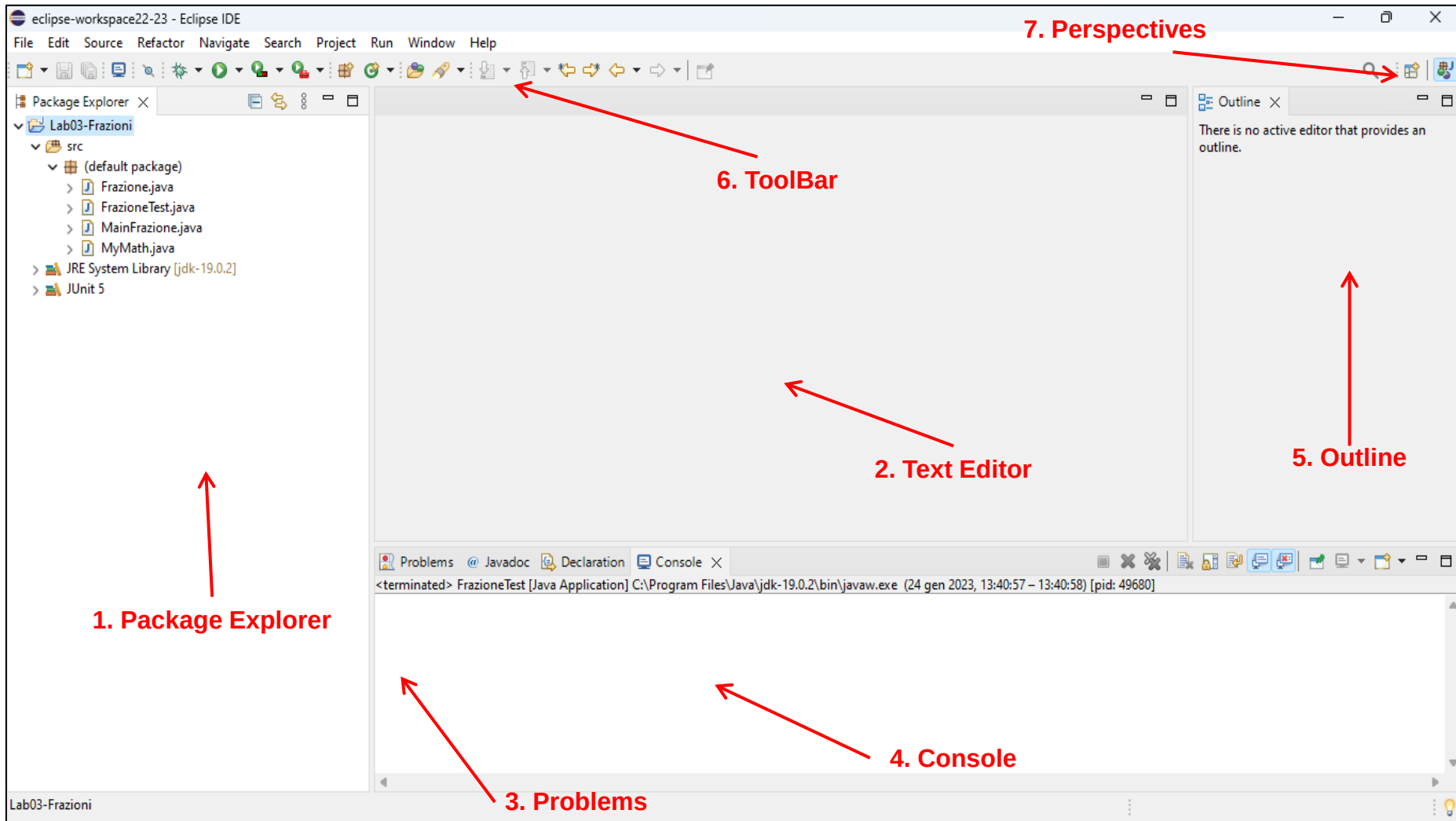
In questo spazio appaiono
editor e toolbar specifiche
di ciò che si sta facendo

OUTLINE: mostra le proprietà
del componente su cui si sta
lavorando

CONSOLE di input/output,
scheda problemi (errori,
warning..), schede informative



Ambiente di lavoro (1/7)

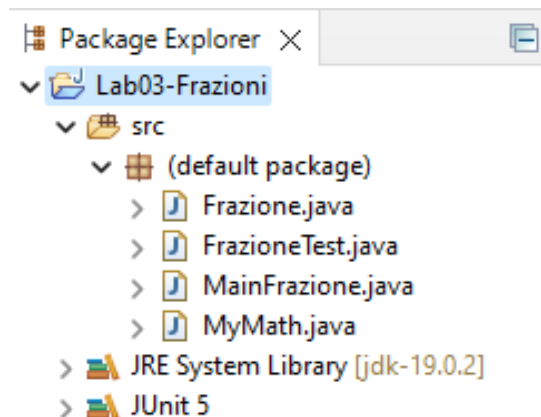




Ambiente di lavoro (2/7)

1. Package Explorer

- Permette di **visualizzare i progetti** all'interno del workspace (tipicamente, un progetto è organizzato in **package**)
- Visualizza le **dipendenze** ad altre **librerie** (ad esempio, dall'infrastruttura base: JRE System Library)
- **i sorgenti di un progetto sono inseriti nella cartella src**





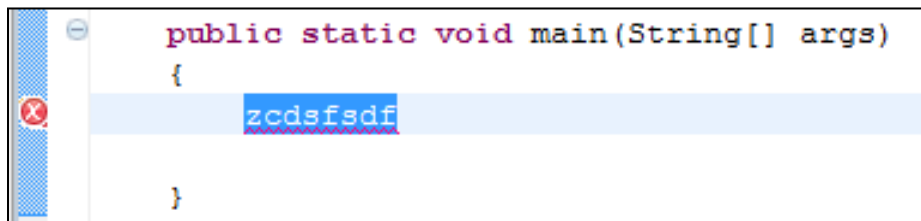
Ambiente di lavoro (3/7)

2. Text Editor

- Permette di scrivere il codice sorgente
- evidenzia con colori differenti le parole chiave e le diverse sezioni del codice

```
/**  
 * @param args  
 */  
public static void main(String[] args)
```

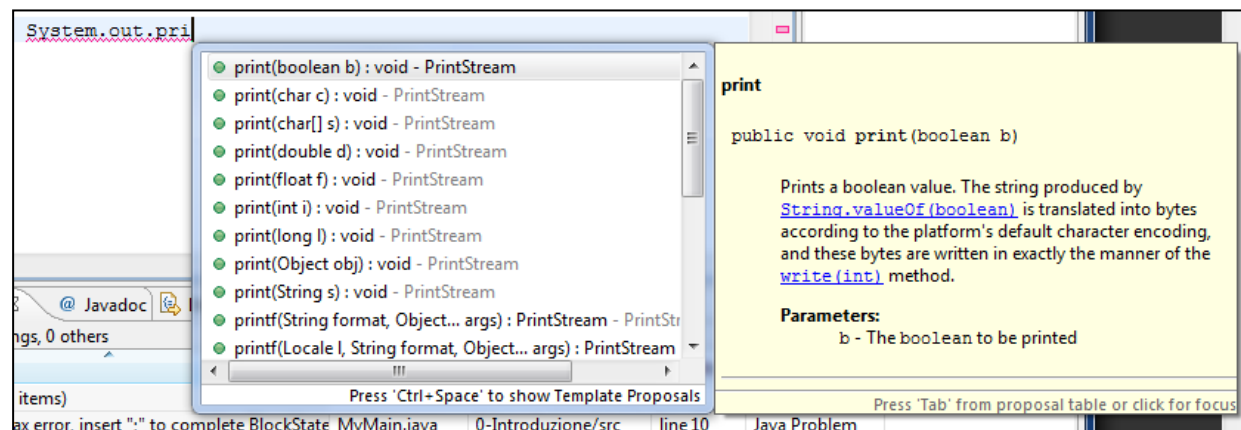
- Eclipse ri-compila automaticamente i sorgenti a ogni modifica, evidenziando eventuali errori



```
public static void main(String[] args)  
{  
    zcdsf sdf  
}
```

Ambiente di lavoro (4/7)

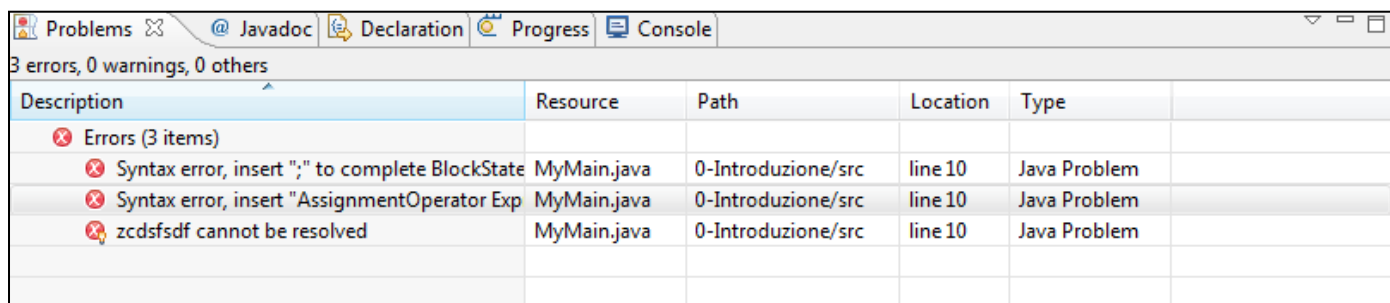
- In caso di errori/warning, cliccando sull'icona rossa/gialla sul lato sinistro del codice in errore/warning viene visualizzato un elenco di probabili cause e relative soluzioni (NON SEMPRE ESATTE...)
- È presente un supporto online che evidenzia i metodi di una classe
Per forzare l'auto completamento → CTRL+SPACE
Non diventate dipendenti dall'autocompletamento! Siate autonomi!
- L'help del metodo selezionato è mostrato nel riquadro



Ambiente di lavoro (5/7)

3. Problems

- Visualizza eventuali **warning (GIALLI)** ed **errori (ROSSI)** di compilazione



The screenshot shows the Eclipse IDE's 'Problems' window. The title bar includes tabs for 'Problems', '@ Javadoc', 'Declaration', 'Progress', and 'Console'. The main area displays '3 errors, 0 warnings, 0 others'. Below this is a table with columns: Description, Resource, Path, Location, and Type.

Description	Resource	Path	Location	Type
Errors (3 items)				
Syntax error, insert ";" to complete BlockState	MyMain.java	0-Introduzione/src	line 10	Java Problem
Syntax error, insert "AssignmentOperator Exp	MyMain.java	0-Introduzione/src	line 10	Java Problem
zcdfsdf cannot be resolved	MyMain.java	0-Introduzione/src	line 10	Java Problem

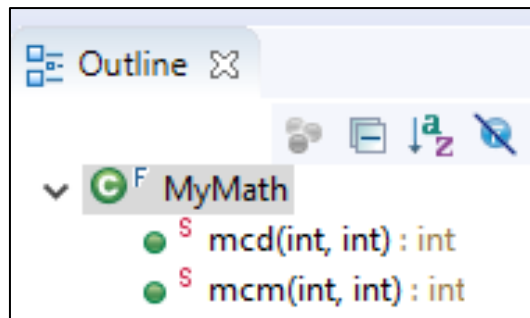
4. Console

- Incapsula la console dell'applicazione:
standard output, standard input, standard error

Ambiente di lavoro (6/7)

5. Outline

- Visualizza la struttura della **classe attualmente visualizzata** nell'editor, attraverso una struttura ad albero



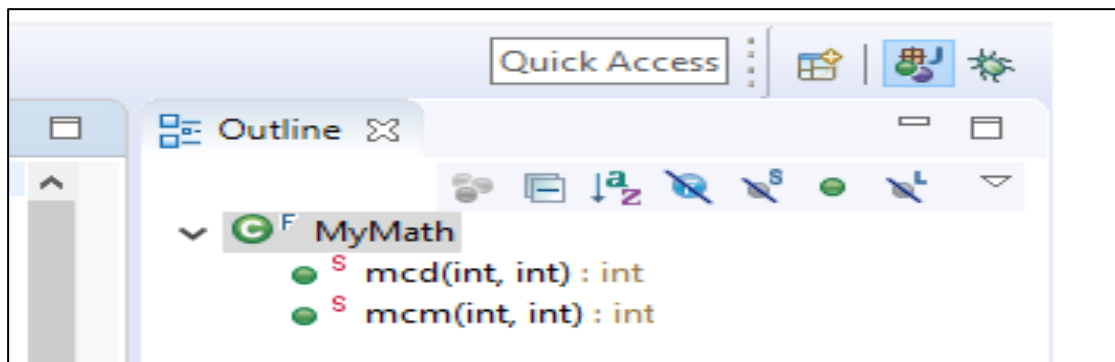
5. ToolBar

- È la classica barra con le operazioni di uso più comune (Apri, Salva, Compila, Esegui, Debug...) personalizzabile

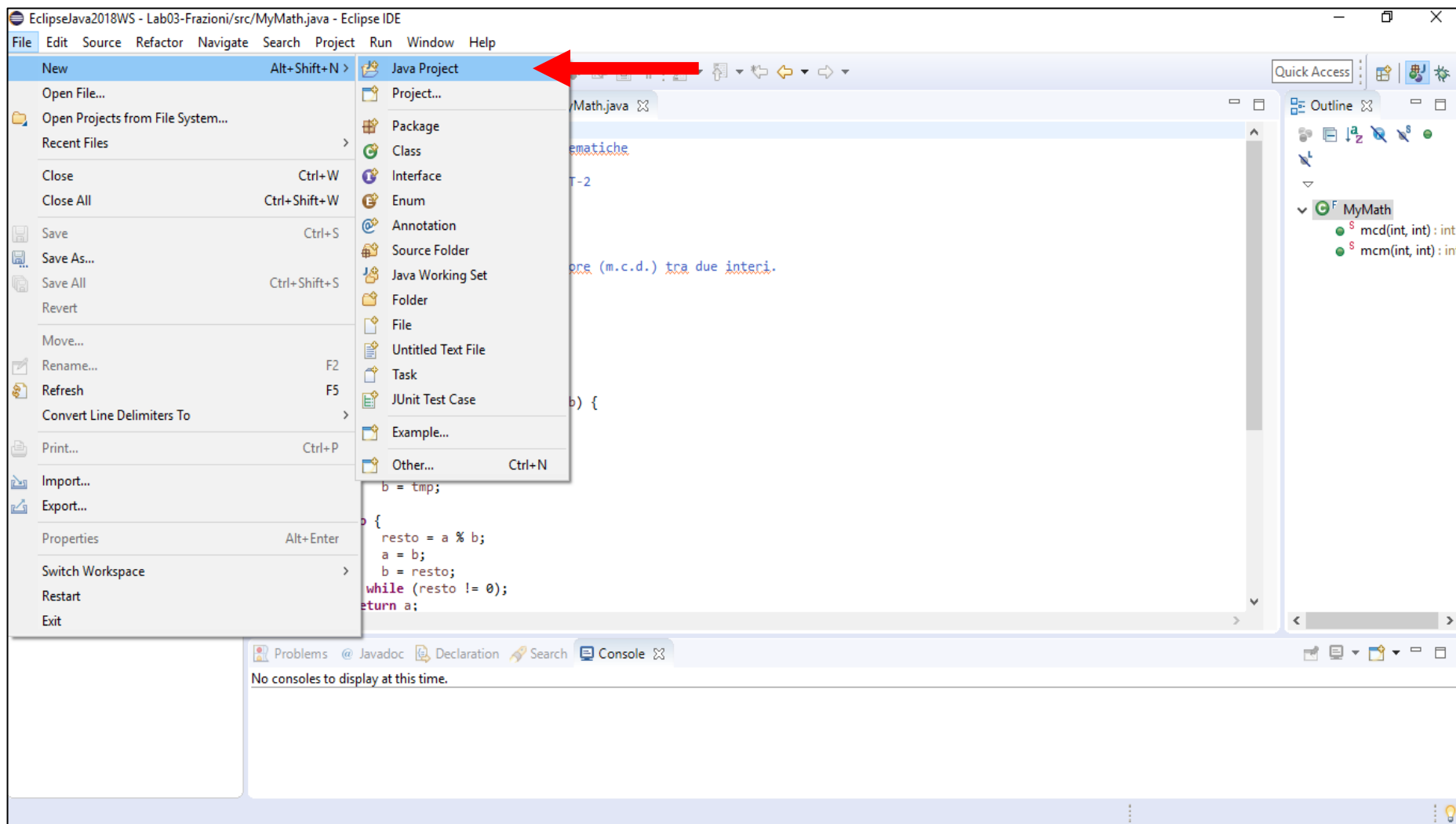
Ambiente di lavoro (7/7)

7. Perspectives

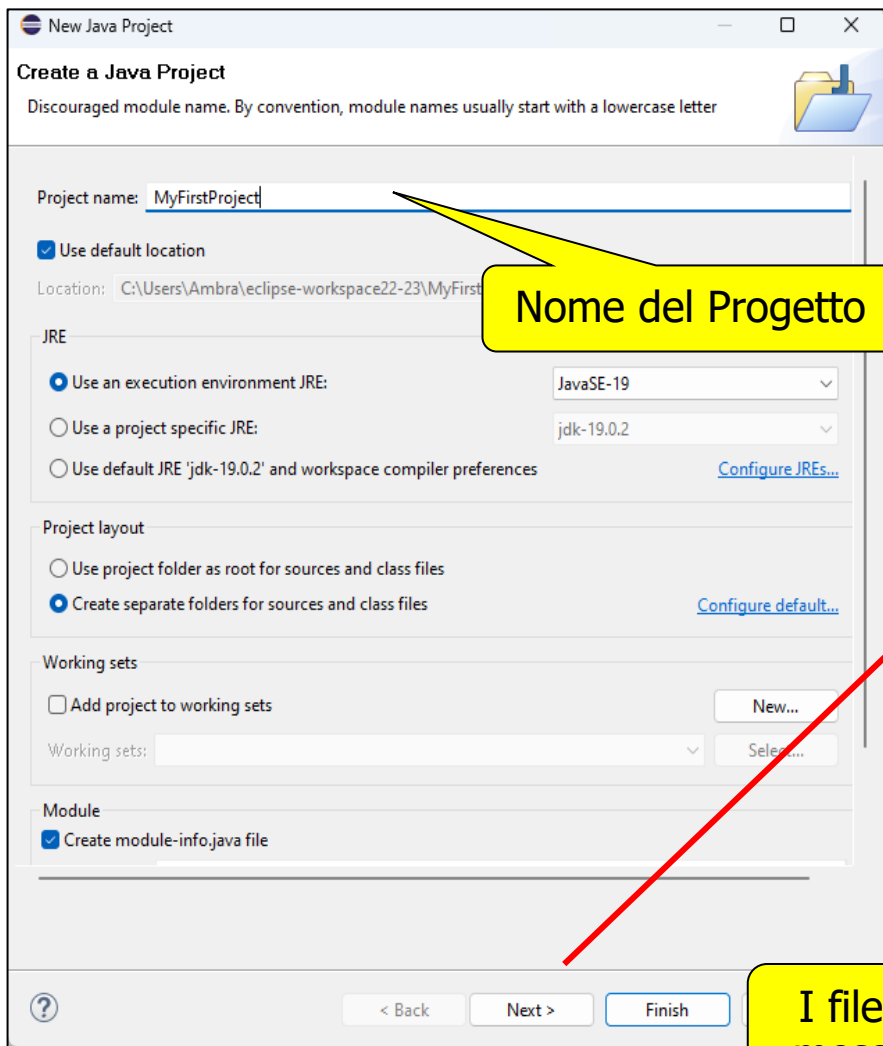
- Eclipse prevede più *prospettive di visualizzazione*
- Ogni prospettiva ha le sue viste e i suoi editor
 - quella mostrata finora è la prospettiva di programmazione
 - *ne esistono altre: ad esempio, quella di debug,* che mostra finestre e strumenti appositi per il debug
 - altre ancora dipendono dai plugin installati



Nuovo progetto (1/2)



Nuovo progetto (2/2)



Create a Java Project

Discouraged module name. By convention, module names usually start with a lowercase letter

Project name:

☒ Use default location

Location: C:\Users\Ambra\workspace22-23\MyFirstProject

JRE

☒ Use an execution environment JRE:

☐ Use a project specific JRE:

☐ Use default JRE 'jdk-19.0.2' and workspace compiler preferences [Configure JREs...](#)

Project layout

☐ Use project folder as root for sources and class files

☒ Create separate folders for sources and class files [Configure default...](#)

Working sets

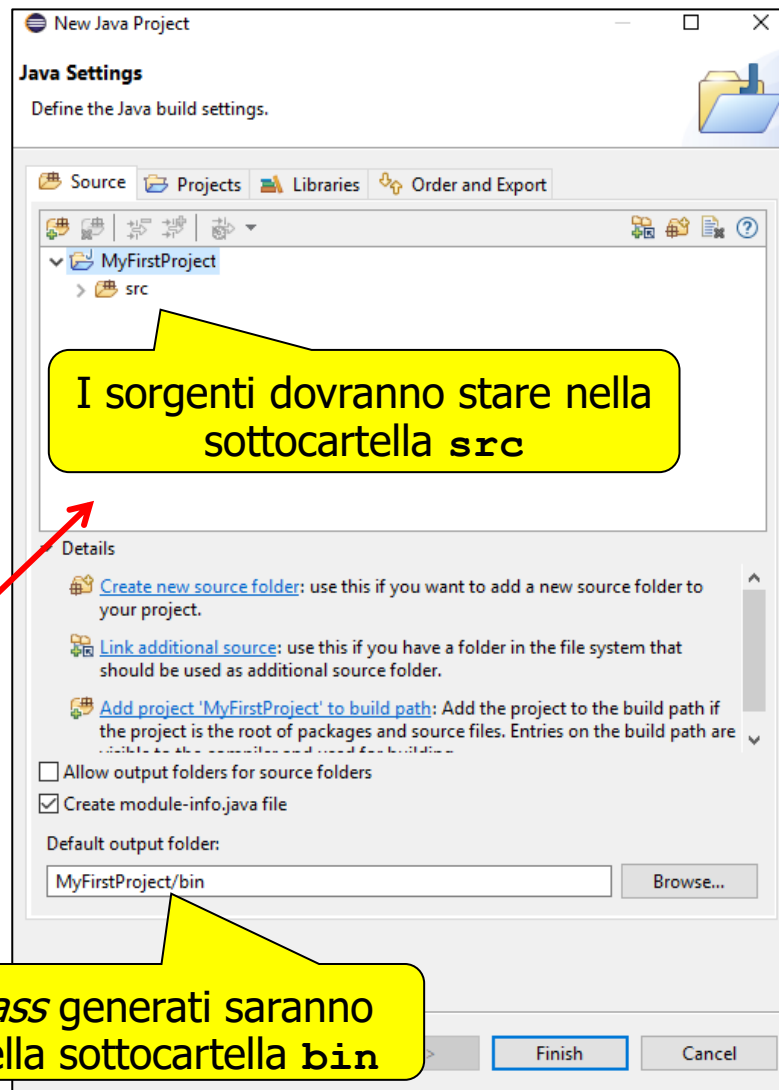
☐ Add project to working sets

Working sets:

Module

☒ Create module-info.java file

Nome del Progetto



Java Settings

Define the Java build settings.

Source **Projects** **Libraries** **Order and Export**

MyFirstProject

- src**

Details

☐ [Create new source folder](#): use this if you want to add a new source folder to your project.

☐ [Link additional source](#): use this if you have a folder in the file system that should be used as additional source folder.

☐ [Add project 'MyFirstProject' to build path](#): Add the project to the build path if the project is the root of packages and source files. Entries on the build path are visible in the **Project Explorer** and used for building.

☐ Allow output folders for source folders

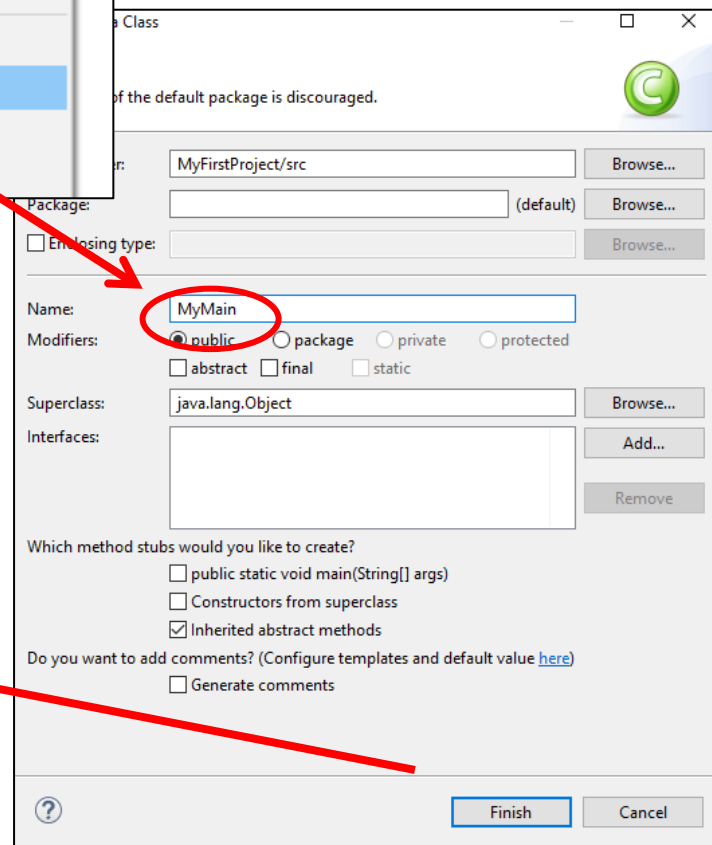
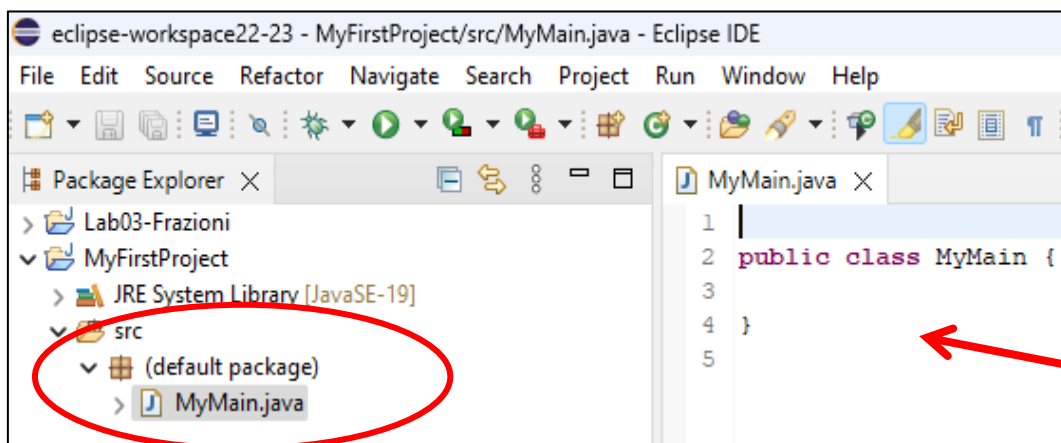
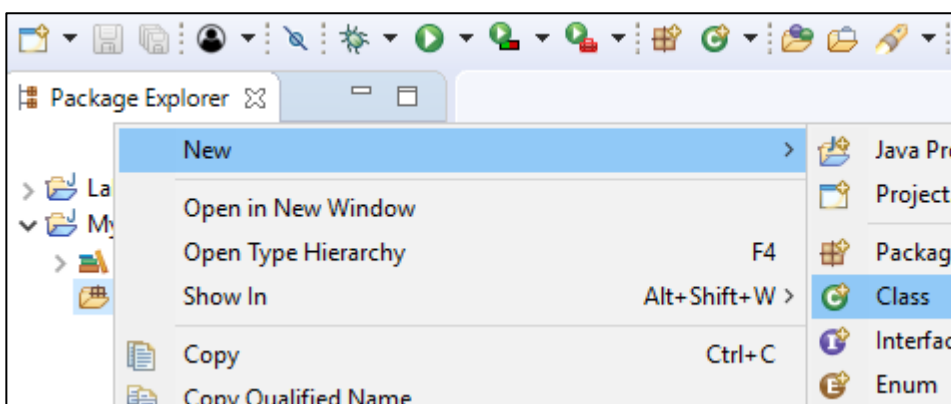
☒ Create module-info.java file

Default output folder:

I sorgenti dovranno stare nella sottocartella **src**

I file **.class** generati saranno messi nella sottocartella **bin**

Il caso base (senza package)



Il caso con package (1/2)

EclipseJava2018WS - MyFirstProject/src/MyMain.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

New
Go Into
Open in New Window
Open Type Hierarchy F4
Show In Alt+Shift+W
Copy Ctrl+C

Java Project
Project...
Package
Class
Interface
Enum

Clic con tasto destro
Scegliere **New > Package**

New Java Package

Java Package
Create a new Java package.

Creates folders corresponding to packages.

Source folder: MyFirstProject/src Browse...

Name: myPackage

☐ Create package-info.java

Nome del Package

Package Explorer

Lab03-Frazioni

MyFirstProject

JRE System Library [JavaSE-19]

src

(default package)

MyMain.java

myPackage

MyMain.java

1
2 public class MyMain
3
4 }
5

Finish Cancel

Il caso con package (2/2)

The screenshot shows the Eclipse IDE interface. On the left, the 'Package Explorer' shows a project named 'MyFirstProject' with a package named 'MyPackage'. A right-click context menu is open over 'MyPackage', showing options like 'New', 'Open in New Window', 'Copy', 'Paste', etc. The 'New' option is selected, and a submenu is visible with 'Java Project', 'Project...', 'Package', 'Class', and 'Interface'. The 'Class' option is highlighted. A yellow callout bubble points to this menu with the text: 'Clic con tasto destro sul package
Scegliere **New > Class**'. On the right, the 'New Java Class' dialog is open. It has fields for 'Source folder:' (MyFirstProject/src), 'Package:' (MyPackage, circled in red), 'Name:' (empty, circled in red), 'Enclosing type:', 'Modifiers:' (public selected), 'Superclass:' (java.lang.Object), and 'Interfaces:'. There are checkboxes for 'Which method stubs would you like to create?' (public static void main, Constructors from superclass, Inherited abstract methods) and 'Do you want to add comments?'. A yellow callout bubble points to the 'Package:' field with the text: 'Nome del package
(e cartella su cui salvare)'. Another yellow callout bubble points to the 'Name:' field with the text: 'Nome della classe
(e relativi modificatori)'. A large yellow callout bubble at the bottom left contains the text: 'Opzionale:' followed by a bulleted list: '• generazione dello scheletro del main', '• generazione dello scheletro dei costruttori (in base a quelli della classe base)', '• generazione dello scheletro dei metodi astratti ereditati dalla classe base'.

Clic con tasto destro sul package
Scegliere **New > Class**

Nome del package
(e cartella su cui salvare)

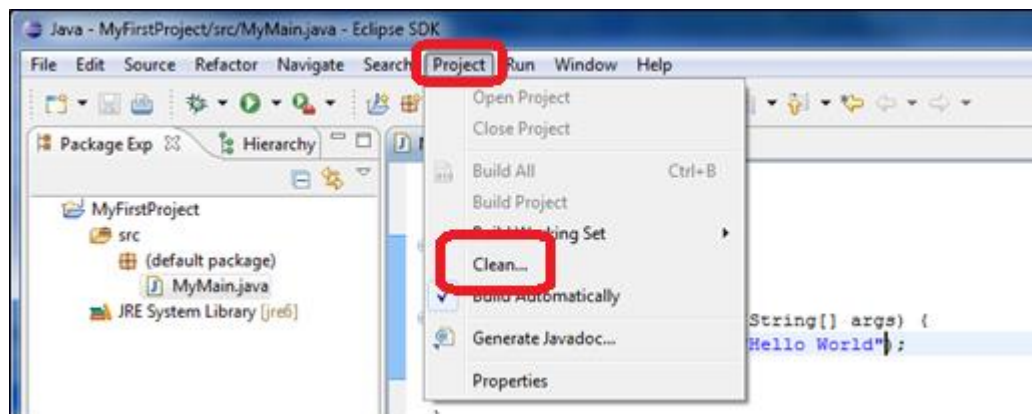
Nome della classe
(e relativi modificatori)

Opzionale:

- generazione dello scheletro del main
- generazione dello scheletro dei costruttori (in base a quelli della classe base)
- generazione dello scheletro dei metodi astratti ereditati dalla classe base

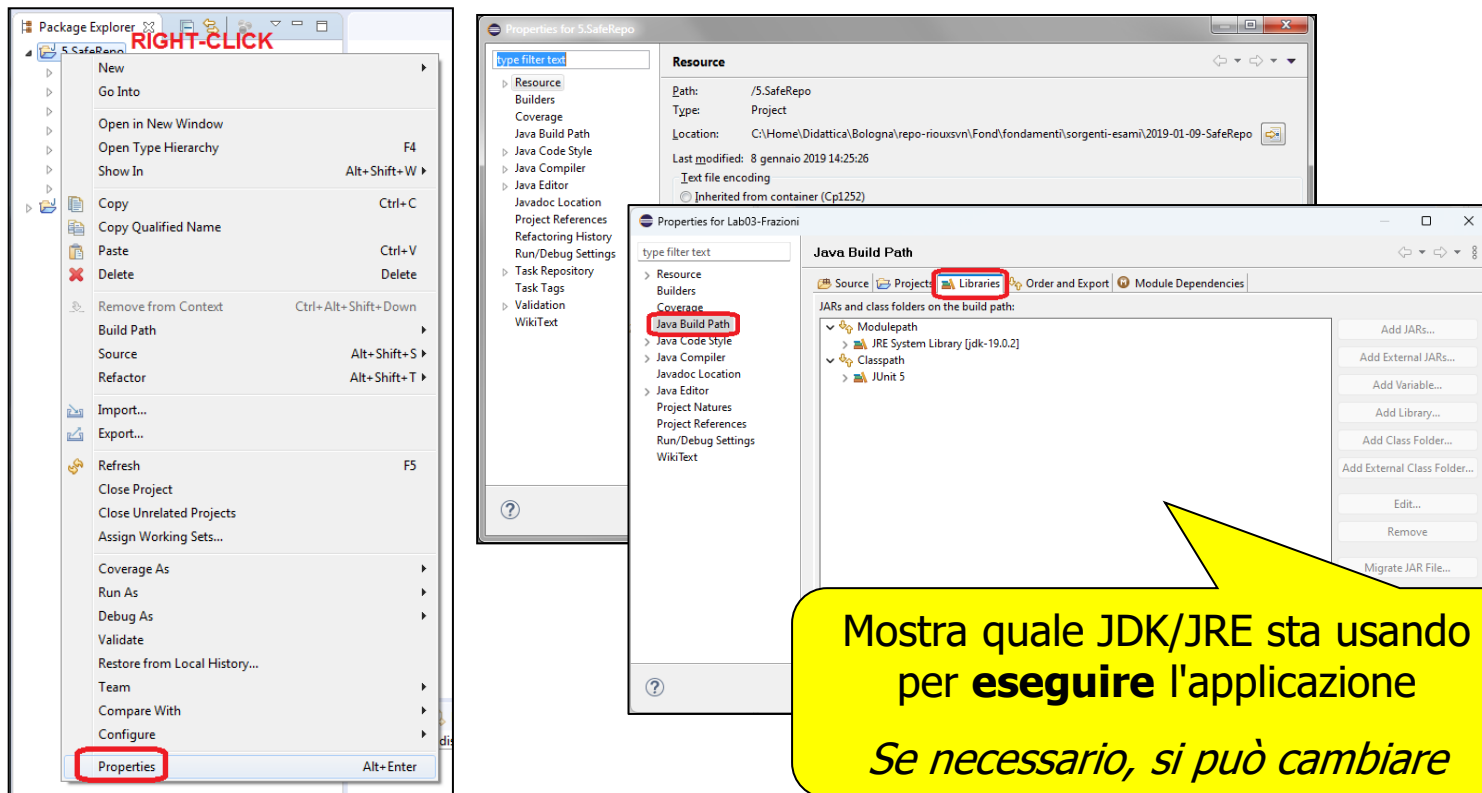
Compilazione

- Come già accennato, Eclipse effettua un *rebuild automatico* ad ogni salvataggio di un file sorgente
 - impostazione *Build Automatically* (attiva di default, ma disattivabile)
 - non è quindi necessario lanciare esplicitamente la compilazione
 - A volte, però, Eclipse non rileva modifiche "accavallatesi" in modo strano → restano errori incomprensibili
- SOLUZIONE:** il comando **Project > Clean**, che elimina tutti i file *.class* forzando una ricompilazione completa



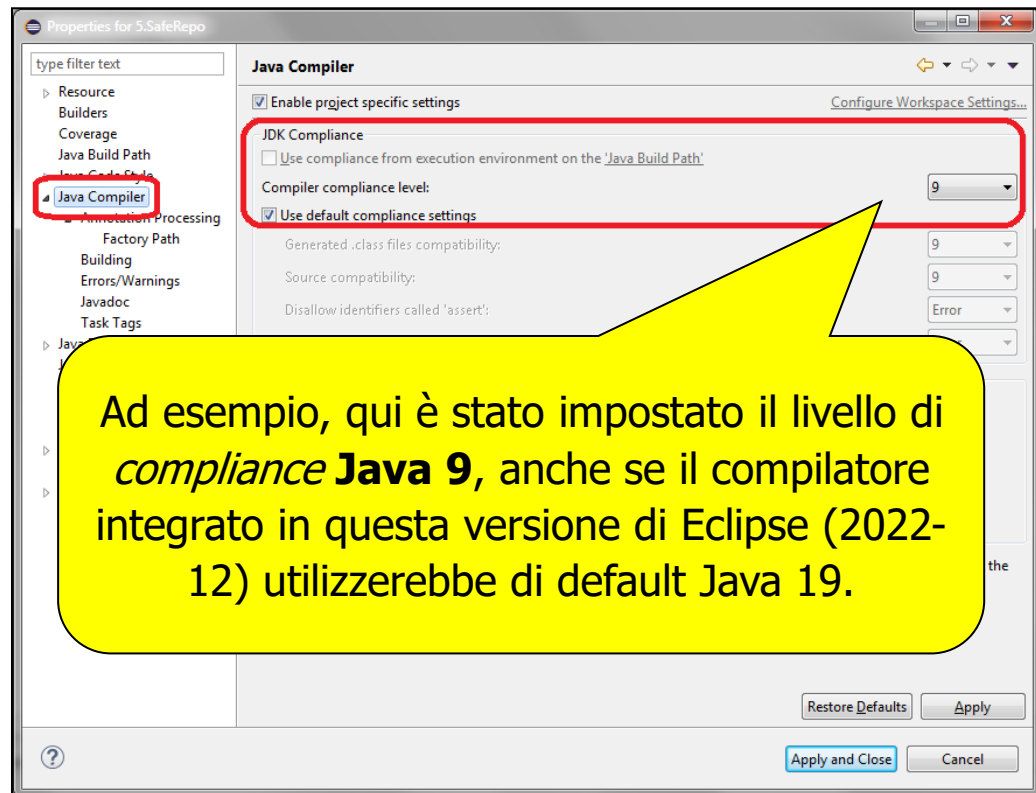
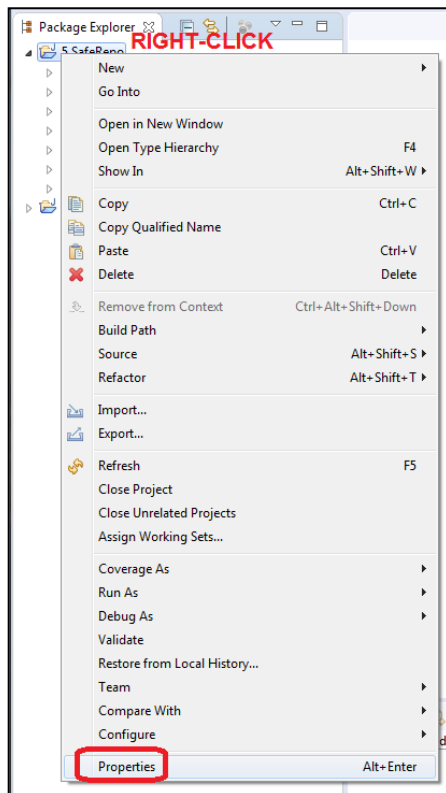
Proprietà del progetto (1/2)

- Talora può essere utile / necessario verificare le **proprietà del progetto**
 - clic con tasto destro sul nome del progetto > *Properties*
 - fra le tante che appaiono, interessa specialmente *Java Build Path*

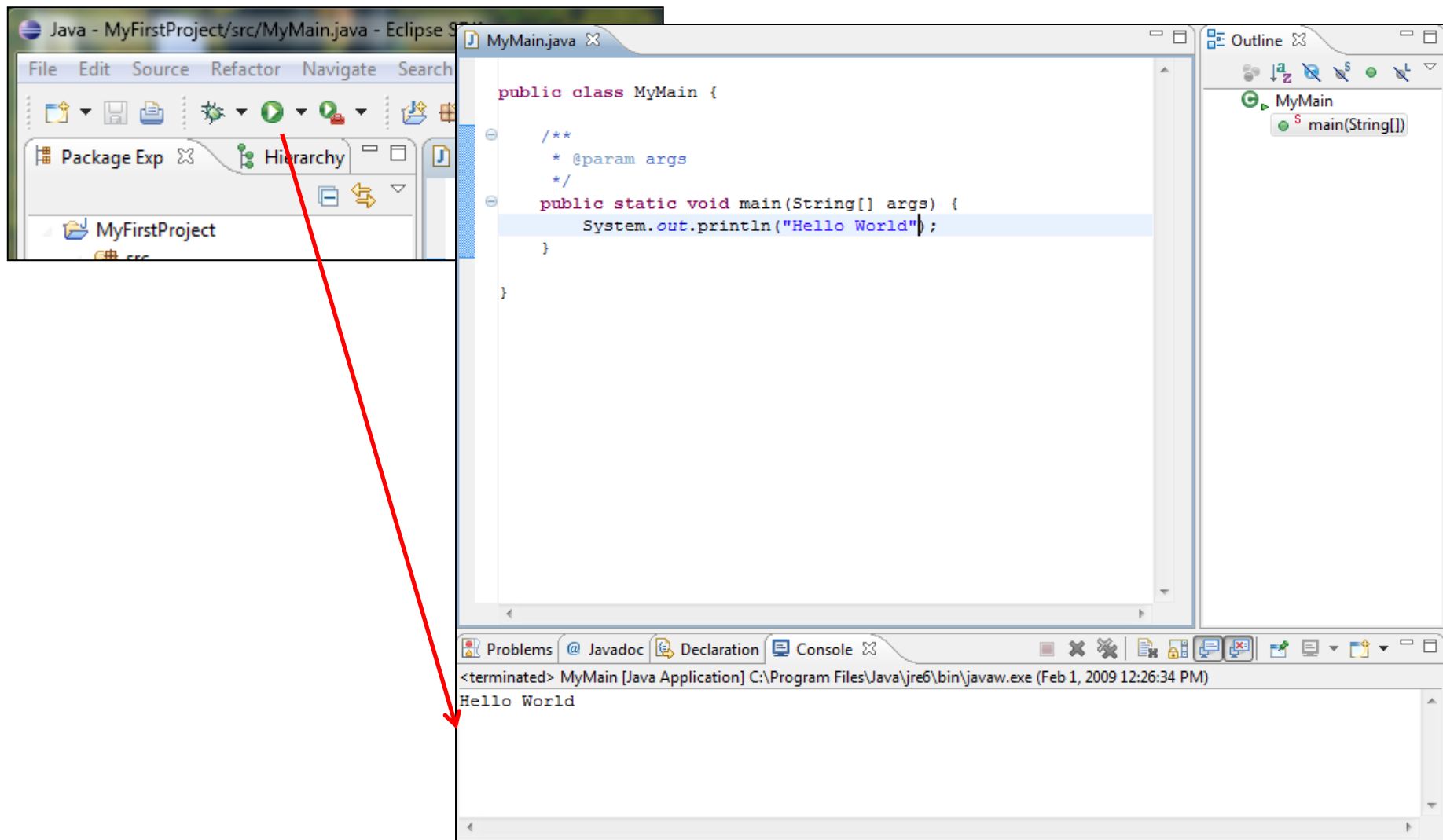


Proprietà del progetto (2/2)

- Se si desidera **compilare per versioni precedenti di Java**
 - ad esempio, compilare per Java 9 pur avendo Eclipse 2022-12, che è Java 19
 - occorre agire sulla proprietà **Java Compiler > JDK Compliance Level**

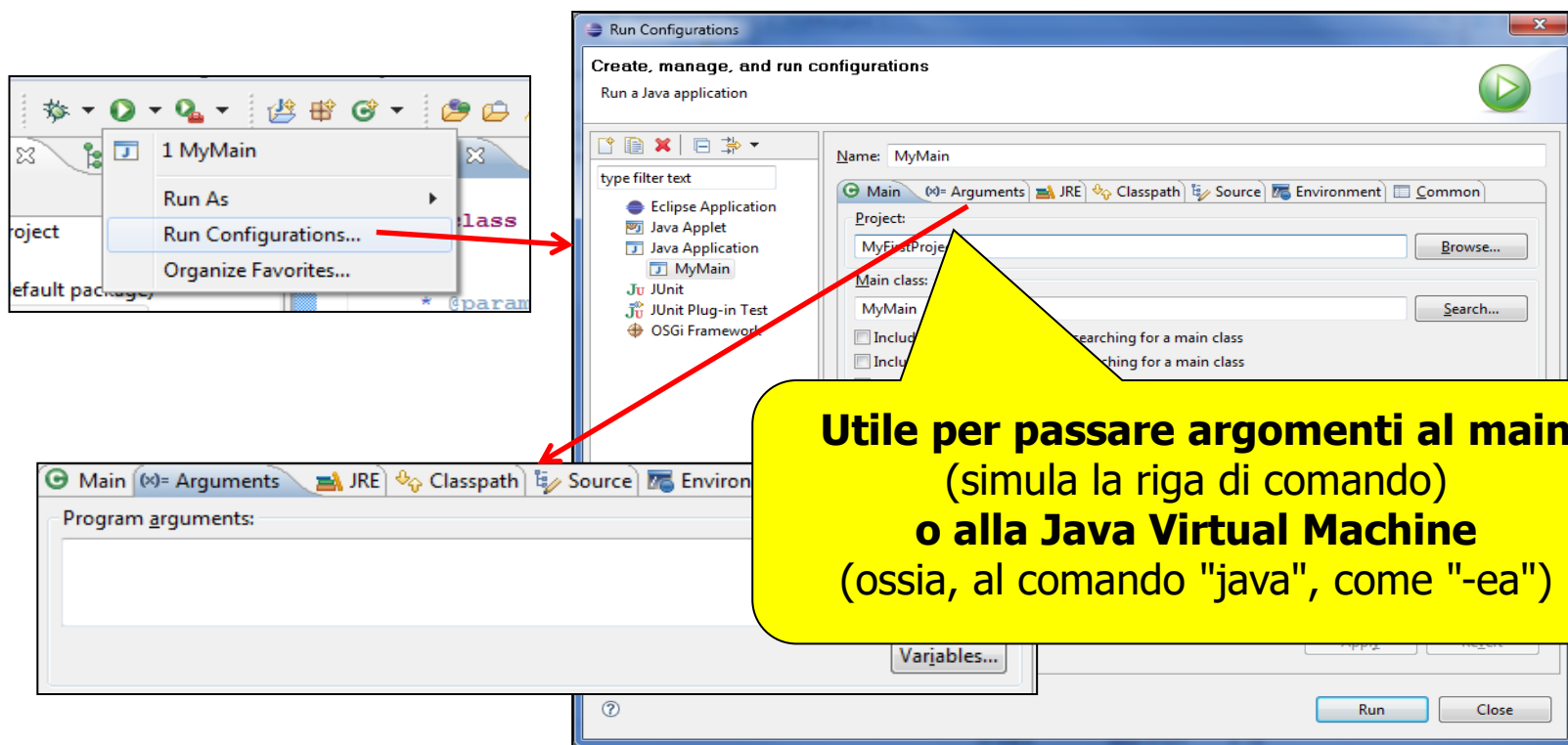


Esecuzione (1/2)

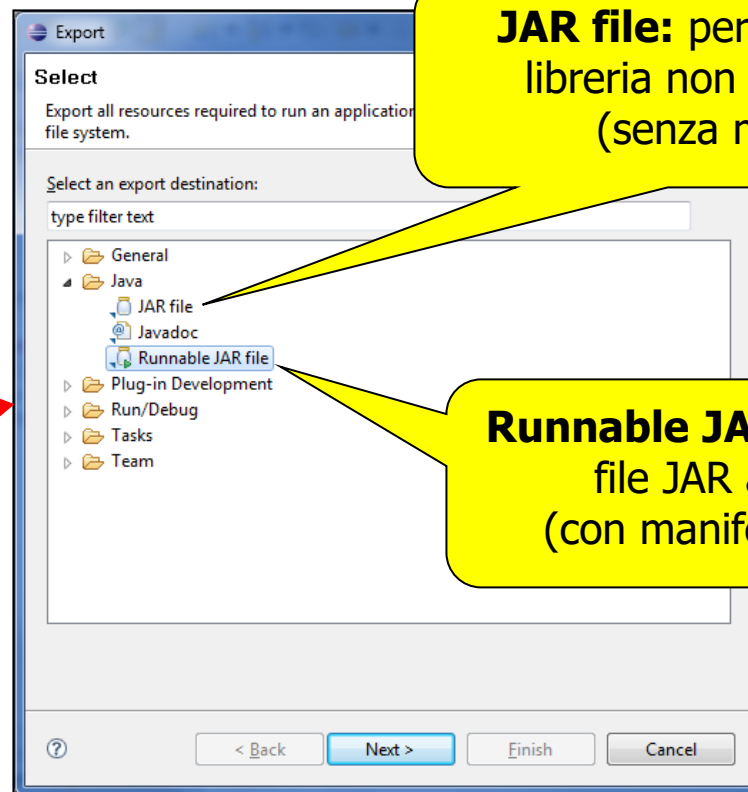
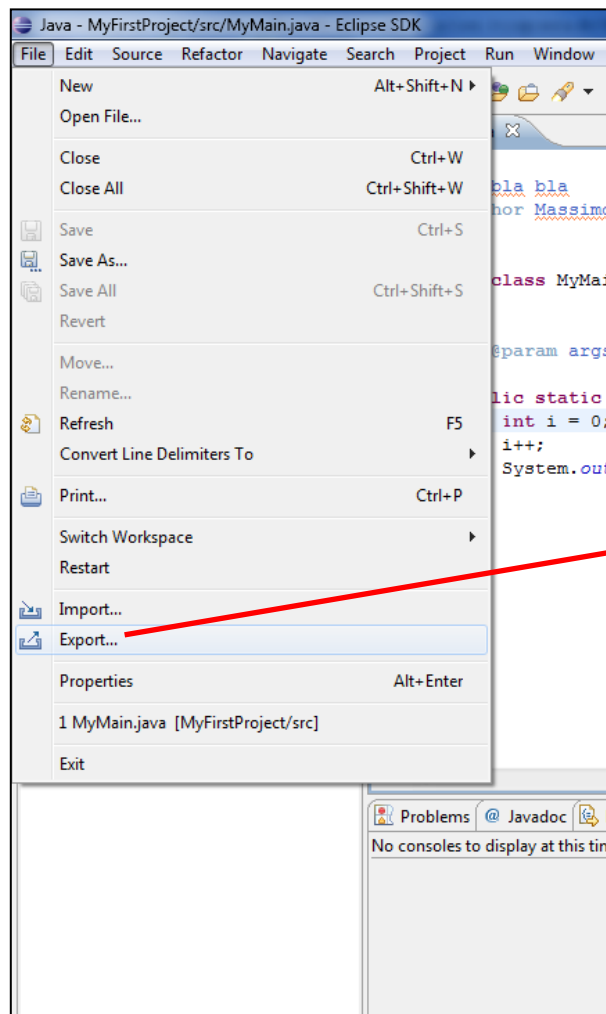


Esecuzione (2/2)

- **Run Configurations** permette di impostare parametri di ingresso, runtime, classpath ecc.
- Si possono **definire più configurazioni** per uno stesso progetto



Creazione di JAR (1/2)

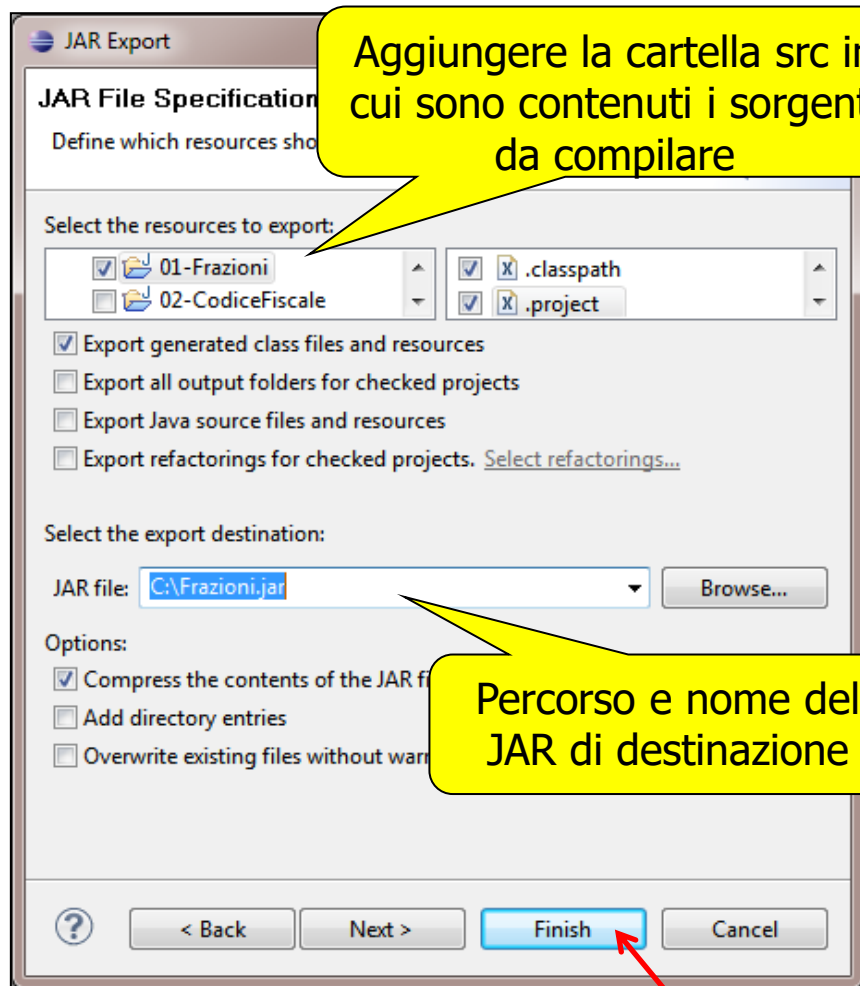


JAR file: per un JAR di libreria non avviabile (senza main)

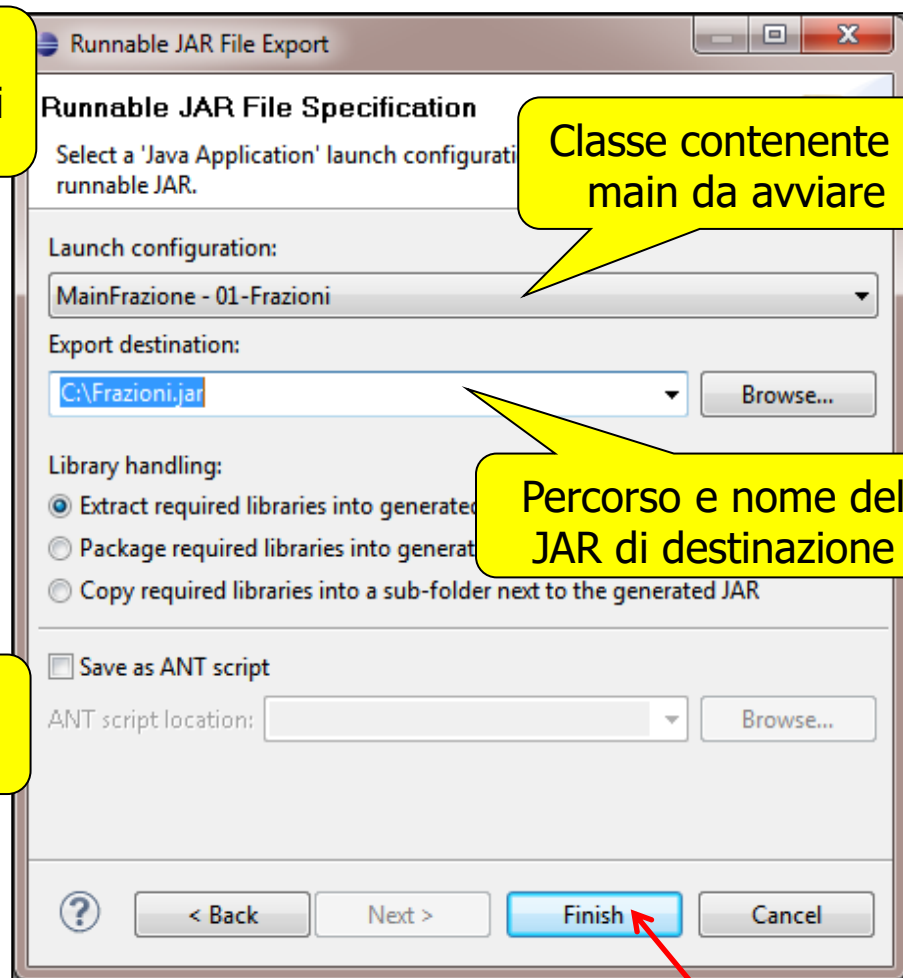
Runnable JAR file: per un file JAR avviabile (con manifest e main)

Creazione di JAR (2/2)

Libreria Jar



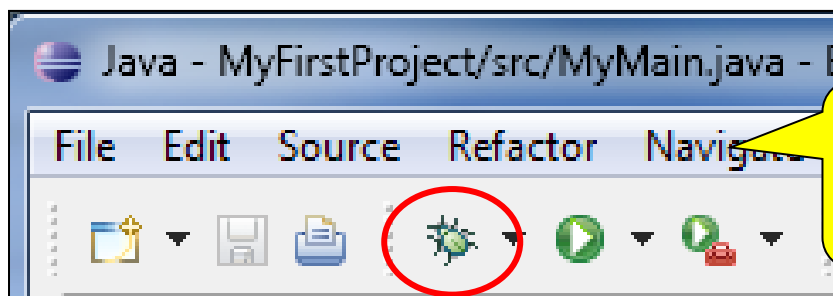
Jar Eseguibile



Debug (1/4)

L'esecuzione dell'applicazione in *debug* permette di:

- eseguire **passo-a-passo**;
- controllare il valore delle **variabili** e vederle evolvere durante l'esecuzione;
- impostare **breakpoint**;
- visualizzare la catena delle chiamate a metodi.



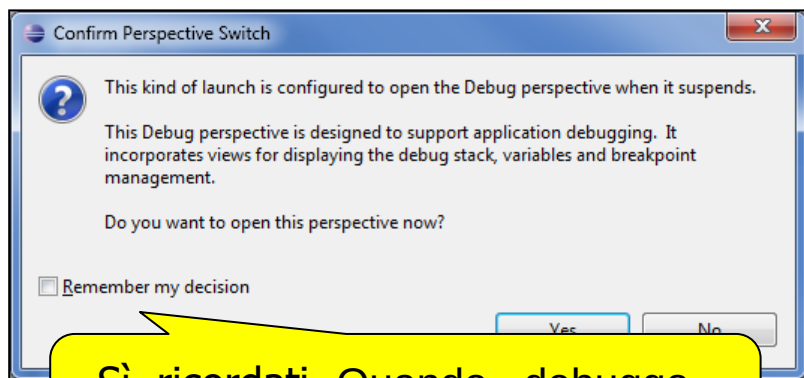
NB: non si può debuggare se non c'è un *main* (o qualcosa del genere...)

Cliccando sull'icona "bug", scatta un **cambio di prospettiva**: dalla **"Java perspective"** alla **"Debug perspective"**.

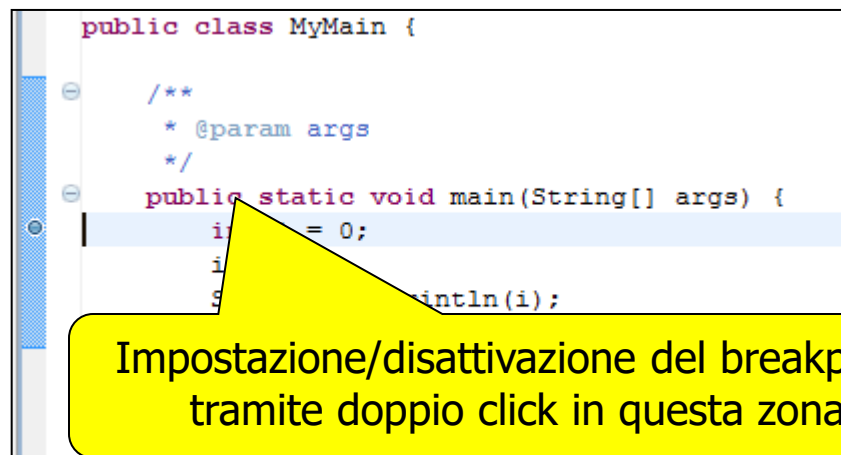
Debug (2/4)

Nel codice sorgente è possibile impostare dei **breakpoint**.

Durante l'esecuzione il debug si fermerà in quel punto.



Sì, ricordati. Quando «debuggo» mi serve la debug perspective.

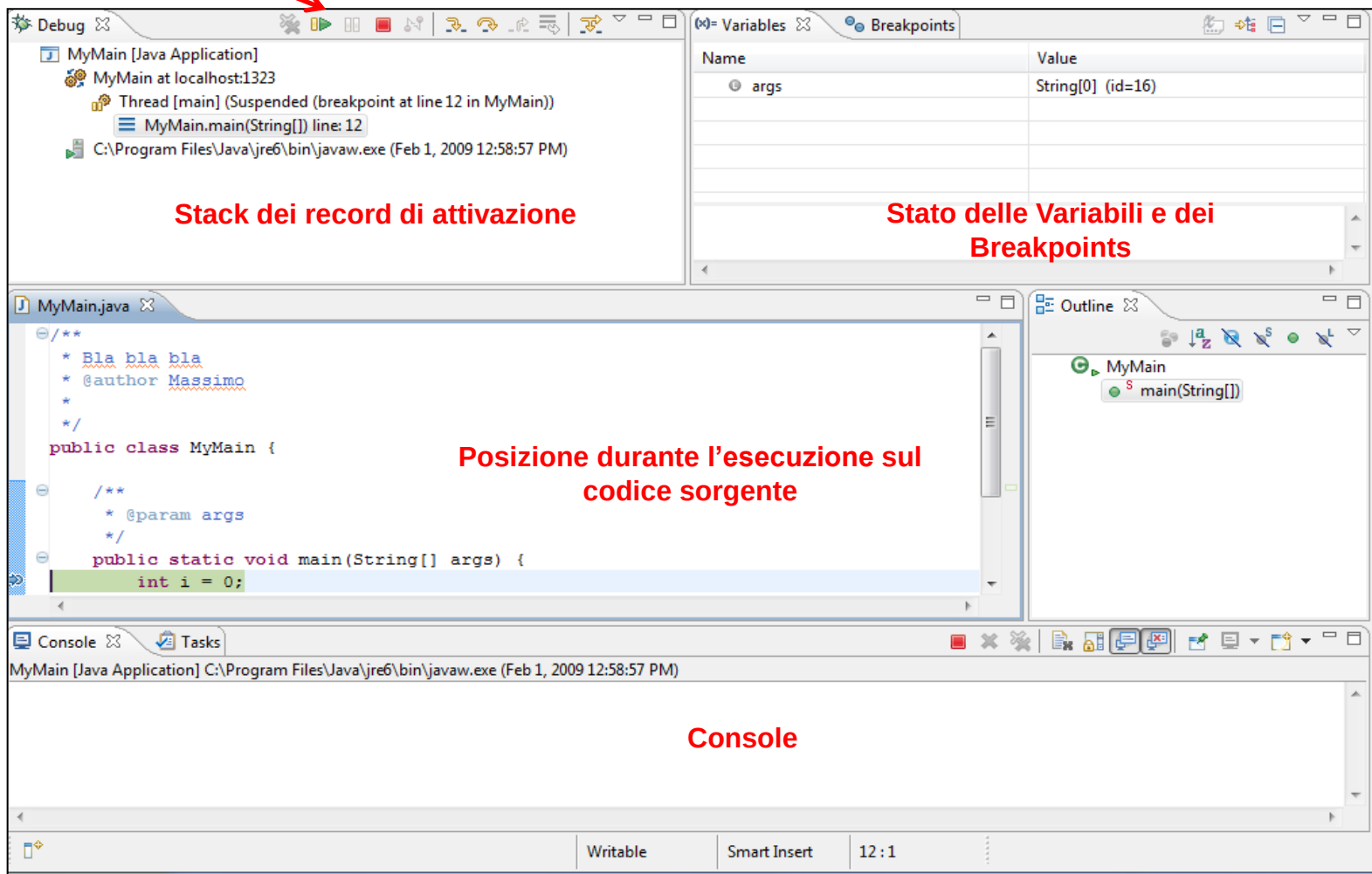


All'avvio del debug verrà richiesto se passare alla **perspective** di debug (...anche sì 😊)

ATTENZIONE: terminato il debug occorre **tornare manualmente** alla *Java perspective*, cliccando sull'apposito bottone.

Debug (3/4)

Menu di Esecuzione



The screenshot shows the Eclipse IDE in Debug mode. The top toolbar contains the execution menu, with a red arrow pointing to the 'Run' button (a green play icon). The 'Debug' console on the left shows the execution stack, including 'MyMain [Java Application]', 'MyMain at localhost:1323', and 'Thread [main] (Suspended (breakpoint at line 12 in MyMain))'. The 'Variables' window on the right shows the state of variables, with 'args' being a 'String[0] (id=16)'. The 'Source' editor in the center shows the code for 'MyMain.java', with the execution position at line 12, 'int i = 0;'. The 'Outline' window on the right shows the project structure, including 'MyMain' and 'main(String[])'. The 'Console' window at the bottom shows the output of the application, including the path 'C:\Program Files\Java\jre6\bin\javaw.exe (Feb 1, 2009 12:58:57 PM)'.

Stack dei record di attivazione

Stato delle Variabili e dei Breakpoints

Posizione durante l'esecuzione sul codice sorgente

Console

Debug (4/4)

Menu Di Esecuzione



→ Continua l'esecuzione fino al breakpoint successivo



→ Mette in pausa l'esecuzione



→ Ferma l'esecuzione

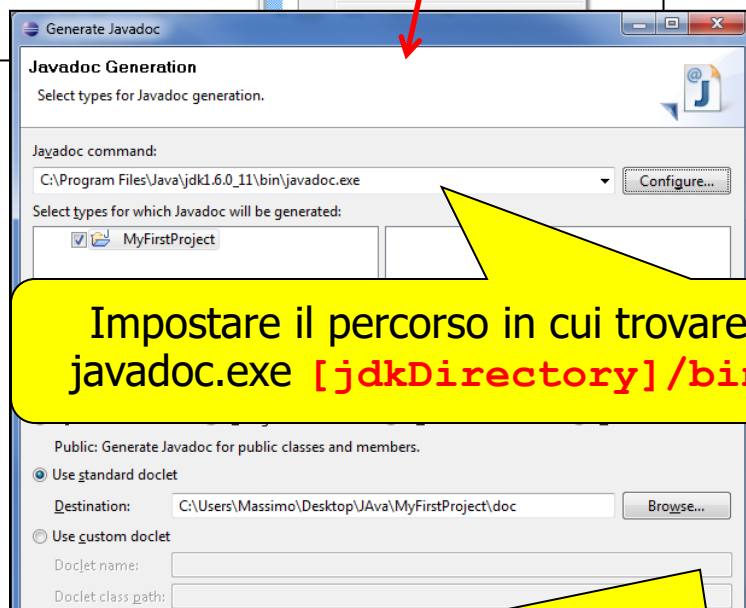
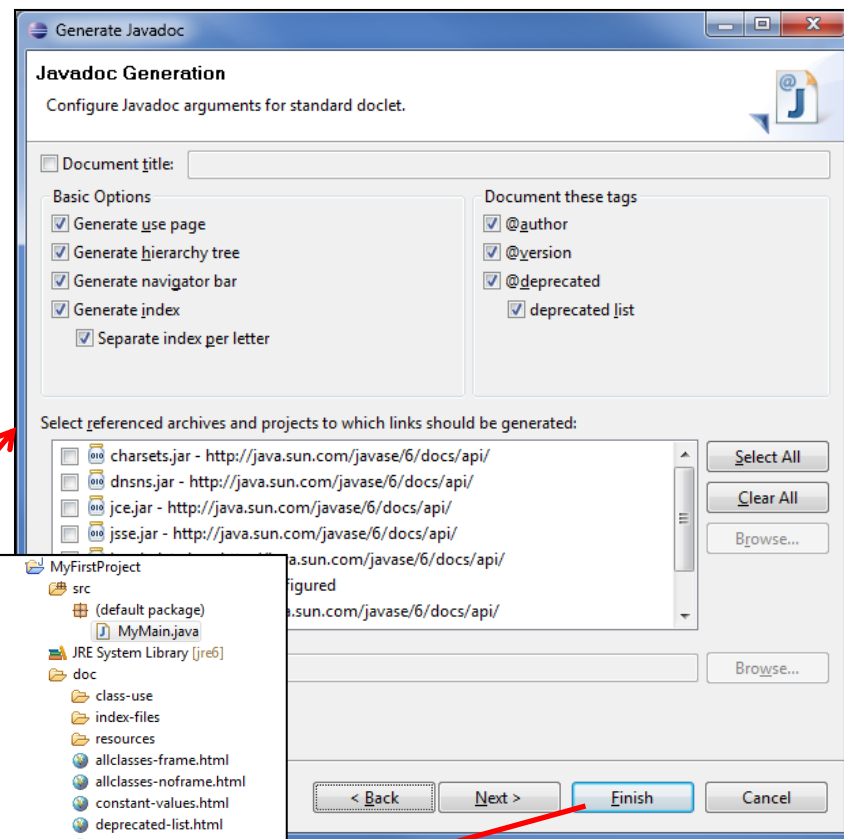
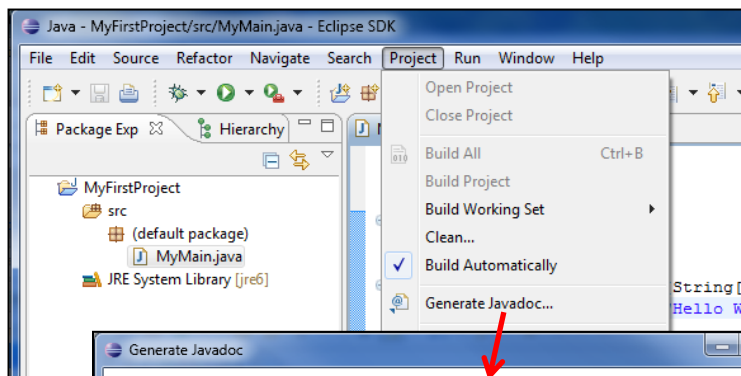


→ Entra all'interno del metodo (shortcut F5)



→ Prosegue senza entrare nel metodo (shortcut F6)

Generazione della documentazione



Impostare il percorso in cui trovare javadoc.exe [jdkDirectory]/bin

Directory in cui generare la documentazione.
Default [projectDirectory]/doc

